

2. **Sequence:** The sequence abstraction shows a way for traversal on sequential views over some set of values —either another collection or values that are the result of some calculation. There are a few operations that sequences undergo:

- `seq` produces a sequence over its argument.
- The first, next and rest of the keywords help make traversal through the `seq` an easy task.

```
user=> (def a (seq '(1 2 3 4 5 6 7)))
#'user/a
user=> (first a)
1
user=>(rest a)
(2 3 4 5 6 7)
```

- It is possible for the contents of a sequence to be evaluated at leisure, where values are computed and produced only when a user tries to access it.
3. **Associative:** The associative abstraction is found in all data structures that map keys to certain values in some way; so, it is mostly seen in various kinds of maps. It has four operations:
- `assoc`, through which connections are made between a key and its value:

```
user=> (def S {:a 10 :b 20 :c 30})
#'user/S
user=> (assoc S :d 40)
{:a 10 :b 20 :c 30 :d 40}
```

- `dissoc`, which cuts off the connections between a key and its value:

```
user=> (def S {:a 10 :b 20 :c 30})
#'user/S
user=> (dissoc S :a )
{:b 20 :c 30}
```

- `get`, which searches the value for a specific key in the collection:

```
user=> (def S {:a 10 :b 20 :c 30})
#'user/S
user=> (get S :a)
10
```

- `contains?`, which checks if a key has a specific value and returns `true` if it does:

```
user=> (def S {:a 10 :b 20 :c 30})
#'user/S
user=> (contains? S :a )
true
```

4. **Indexed:** The index abstraction here is similar to the one in C++. The index can be given to access an element present at that specific position. There is only one function here, `nth`, which acts like the `get` function and returns the element at the `nth` position. There are also exceptions to check the out-of-bounds condition.

```
user=> (def S '[1 2 3 4 5])
#'user/S
user=> (nth a 2)
3
```

5. **Stack:** Clojure does not have a stack data structure as such, but we can use lists and vectors to realise this data structure. The three main operations here are:

- `conj`, to add an element to the top of the stack:

```
user=> (def stack '(5 4 3 2 1))
#'user/stack
user=> (conj stack 6)
(6 5 4 3 2 1)
```

- `pop`, to remove the element present at the top of the stack:

```
user=> (def stack '(5 4 3 2 1))
#'user/stack
user=> (pop stack )
(4 3 2 1)
```

- `peek`, for obtaining the value on the top of the stack:

```
user=> (def stack '(5 4 3 2 1))
#'user/stack
user=> (peek stack)
5
```

6. **Set:** The set abstraction is technically a reusing of the functions in a collection as well as associative semantics. The `clojure.set` namespace provides a number of functions that can be performed on a set, including `subset?`, `superset?`, `union`, etc.

```
user=> (require '[clojure.set :as t])
nil
user=> (def a '[1 2 3 4 5 6 7 8])
#'user/a
user=>(def b '[1 2 3])
#'user/b
user=>(t/subset? b a)
true
user=>(t/superset? a b)
true
```

Continued on page no 68...